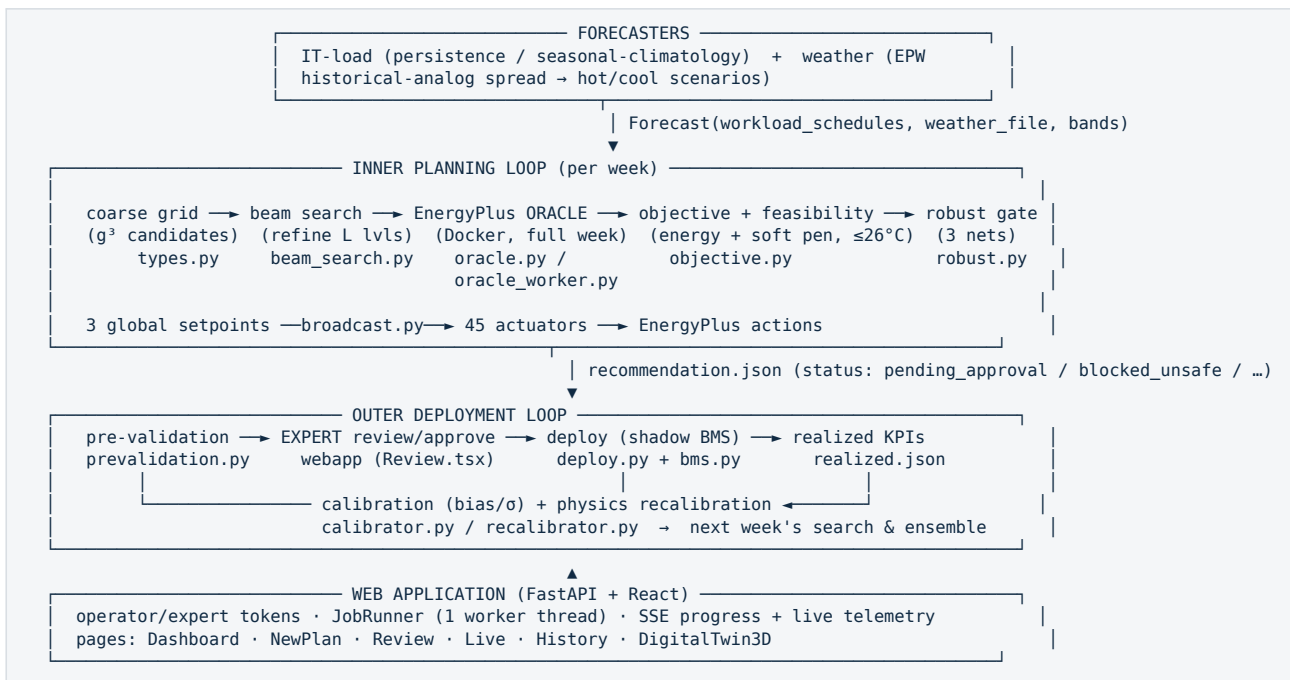


DCTwin User & Developer Guide

DCTwin is a **digital-twin dual-loop optimizer for weekly data-center cooling setpoints**. Each week it searches three global setpoints — CRAH supply-air temperature (**SAT**, 20–26 °C), CRAH airflow (**flow**, 4.8–13.8 kg/s), and chilled-water supply temperature (**CHWST**, 13–19 °C) — and scores every candidate with a **full-week EnergyPlus 9.5 run executed in Docker** (the *oracle*), not a surrogate model. The 3 global setpoints are broadcast to **45 actuators** (22 SAT + 22 FLOW + 1 CHWST) for the controlled 1F-2A hall. A **hard safety invariant — rack inlet temperature ≤ 26 °C at every timestep — is non-negotiable** and enforced by a layered "three-net" safety gate. An inner *planning loop* (forecast → beam search → oracle → robust gate → recommendation) proposes a plan; an outer *deployment loop* (pre-validate → expert approve → deploy → calibrate) closes the fidelity gap and feeds learning back into the next week.

System at a glance



The two loops share one EnergyPlus oracle and one calibration state. The web app is the human-facing skin over both loops; the forecasters supply the workload + weather inputs the oracle simulates against.

Repository layout

Path	Purpose
<code>src/planner/</code>	The inner planning loop: beam search, oracle, objective, robust gate, forecasters, calibration, baseline, broadcast, schedule.
<code>src/webapp/</code>	FastAPI backend: routes (<code>main.py</code>), job runner (<code>jobs.py</code>), SQLite store (<code>store.py</code>), auth (<code>auth.py</code>), status state machine (<code>status.py</code>), telemetry (<code>telemetry.py</code>), 3D topology (<code>topology.py</code>).

Path	Purpose
<code>src/frontend/</code>	React 19 + Vite + TypeScript UI. <code>src/pages/</code> (Dashboard, NewPlan, Review, Live, History, DigitalTwin3D), <code>src/three/</code> (3D scene), <code>src/api.ts</code> (client).
<code>src/*.py</code> (CLI)	<code>plan_weekly.py</code> , <code>fit_forecaster.py</code> , <code>prevalidation.py</code> , <code>deploy.py</code> , <code>fit_recirc.py</code> , <code>backtest_forecaster.py</code> .
<code>dctwin/</code>	The EnergyPlus/BCVTB co-simulation engine: <code>registration.py</code> (<code>make_env</code>), <code>gym_envs/eplusr_env.py</code> , <code>third_parties/eplusr/core.py</code> , <code>third_parties/docker_backend.py</code> .
<code>src/runs/</code>	Per-week result artifacts (<code>gds-2024-MM-DD-HASH/</code>): <code>recommendation.json</code> , <code>realized.json</code> , <code>oracle/</code> , <code>robust/</code> , <code>prevalidation/</code> , <code>deploy/</code> , <code>plant/</code> .
<code>src/log/</code>	Operational logs (planning, oracle, diagnostics, sweeps, screenshots, <code>backend.out</code>).
<code>models/</code> , <code>configs/</code> , <code>data/</code>	Planner assets (gitignored): <code>forecaster.pkl</code> , <code>dt.prototxt</code> , <code>calibration.json</code> , EPW weather, <code>his_data_processed.csv</code> , etc. Copy the GDS model from <code>mycode/Tropical_DC_Files/GDS_Nov_Supply_Return32_CHWT_Backup</code> .
<code>docs/superpowers/specs/</code> & <code>docs/superpowers/plans/</code>	Design specs + TDD implementation plans (dated spec+plan pairs per shipped tier).
<code>graphify-out/</code>	AST knowledge graph of <code>src/</code> (<code>GRAPH_REPORT.md</code> , <code>graph.html</code> , <code>graph.json</code>).
<code>dctwin/BHUMANS/</code>	Report assets: figures (<code>deck_assets/figA...figF.png</code>), decks, manuscript, review notes.
<code>scripts/clear-and-run.sh</code>	One-command launcher (clears state, builds frontend, serves the whole app).

Quick start (operators)

Environment

DCTwin uses a non-standard virtualenv and a sandboxed shell. Two rules:

- **Python interpreter:** always `/mnt/lv/home/hoanghuy/newcode/dctwin/.venv-dctwin/bin/python`. There is no `python/python3` on PATH with the deps. The backend shares this venv so it can run real EnergyPlus plans.
- **cd quirk:** a leading `cd` is stripped from shell commands. Use `env -C <dir> ...` or `git -C <repo> ...` instead.
- The Python package roots are under `src/` (from `planner...`, from `webapp...`). Run from `src/` or set `PYTHONPATH=.../src`.
- **Docker** is needed only to *run* a plan (it launches EnergyPlus). Wrap those commands in `sg docker -c "..."`.

Step 1 — Fit the forecaster (once, offline)

A fitted forecaster is **required before any plan**. It builds the room→column mapping and pickles a config to `models/forecaster.pkl`.

```
env -C /mnt/lv/home/hoanghuy/newcode/dctwin/src \
/mnt/lv/home/hoanghuy/newcode/dctwin/.venv-dtwin/bin/python fit_forecaster.py
```

To thread a weather file and pick the seasonal method:

```
env -C /mnt/lv/home/hoanghuy/newcode/dctwin/src \
/mnt/lv/home/hoanghuy/newcode/dctwin/.venv-dtwin/bin/python -c \
"import fit_forecaster; fit_forecaster.main(weather_file='data/weather/Singapore_Changi_Nov2024-Jan2025.epw')"
```

Step 2 — Run the web app

The simplest path clears plan state, builds the UI, and serves the **whole app from one origin** at <http://localhost:8000> (UI at [/](#), API at [/api/*](#)):

```
scripts/clear-and-run.sh          # → http://localhost:8000 (single origin)
scripts/clear-and-run.sh --dev    # → http://localhost:5173 (Vite hot reload + backend proxy)
```

To start the backend by hand (Docker group, both role tokens):

```
sg docker -c "env -C /mnt/lv/home/hoanghuy/newcode/dctwin/src PYTHONPATH=\$PWD \
  OPERATOR_TOKEN=op EXPERT_TOKEN=ex \
  /mnt/lv/home/hoanghuy/newcode/dctwin/.venv-dtwin/bin/python -m uvicorn webapp.main:app --port 8000"
```

`webapp/main.py` mounts `frontend/dist` at `/` only when it's built — so run `npm --prefix src/frontend run build` (or `clear-and-run.sh` does it for you), or `/` shows a "not built" hint instead of the UI.

Step 3 — The end-to-end operator workflow

#	Action	Who	UI page / endpoint
1	Paste token, log in	operator/expert	Login → GET /api/plans (verify)
2	Create a new weekly plan (<code>week_start</code> , <code>days</code> , search params)	operator	NewPlan → POST /api/plans
3	Watch live search progress (level, evals, best score)	operator	NewPlan → SSE GET /api/plans/{id}/stream
4	Review setpoints, predicted KPIs, baseline comparison, robust bands, trajectory	expert	Review → GET /api/plans/{id}
5	(optional) Edit setpoints — invalidates prevalidation, blocks approval	expert	Review → PATCH /api/plans/{id}/setpoints
6	Approve (or reject)	expert	Review → POST /api/plans/{id}/approve
7	Deploy the approved plan (shadow mode)	expert	Review → POST /api/plans/{id}/deploy
8	Monitor live racks / power / compliance	operator	Live → GET /api/live , SSE /api/live/stream

The equivalent headless CLI flow (from [src/README.md](#)):

```
env -C /mnt/lv/home/hoanghuy/newcode/dctwin/src ... python plan_weekly.py --week-start 2024-11-11
env -C ... python prevalidation.py --recommendation runs/<id>/recommendation.json          # validate
env -C ... python prevalidation.py --recommendation runs/<id>/recommendation.json --approve # expert approve
# deploy: deploy(rec_path, ParallelEnvOracle(...)) — runs the plant week, records realized KPIs
```

How to read results

- **Predicted vs realized energy:** `recommendation.json` → `predicted_kpis` vs `realized.json` → `total_hvac_energy_kwh`. The week's prediction error is the gap.
- **Savings:** `energy_reduction_vs_baseline_pct` in `predicted_kpis` = $(\text{baseline_kwh} - \text{plan_kwh}) / \text{baseline_kwh} \times 100$.
- **Safety:** `realized.json` → `inlet_temp_max_c` (must be ≤ 26.0) and `inlet_violation_steps` (must be 0). Any violation flips deploy status to `deploy_blocked`.
- **Status:** `recommendation.json` → `status` is the plan's place in the state machine (see [Web application](#)).

The inner planning loop

The inner loop turns a `PlanRequest` + forecast into a `recommendation.json`. Its core orchestrator is `run_weekly_plan()` (`src/planner/pipeline.py:62`), which builds a `BeamPlanner`, runs the search, applies the calibration forecast margin, optionally reranks finalists through the robust ensemble, and falls back to the safety ladder when the optimum is fragile.

5.1 Beam search, objective & constraints

What it does. A best-first **coarse-to-fine** search over the 3D setpoint space. Level 0 evaluates a `grid3` coarse grid; subsequent levels generate local neighbors around each beam node with a halving step. Every candidate is scored against a single scalar objective with a hard feasibility gate.

The objective (`src/planner/objective.py:47`, minimized over feasible candidates):

```
score(kpi) = energy_term
            + λ_temp · inlet_excess_degc_steps      (default λ_temp = 1.0)
            + λ_rh   · rh_excursion_steps          (default λ_rh   = 0.2)
            + λ_zone · zone_temp_band_steps        (default λ_zone = 0.1)
            = INFEASIBLE (math.inf) if not is_feasible(kpi)
```

`energy_term` is `kpi.total_hvac_energy_kwh`, or `kpi.weighted_energy_cost` when a tariff is loaded (the two never combine — it's pure kWh or pure cost/carbon). The soft-penalty accumulators are computed **by the oracle**, not the objective; the objective only sums them. Infeasible candidates score `+inf` and sink to the bottom of the stable sort, so they can never enter the beam (`_top_b`, `beam_search.py:170`).

The hard feasibility gate `is_feasible()` (`objective.py:35`) is conjunctive — **all** must hold:

- 1 `kpi.feasible == True` (simulator converged, no physical error).
- 2 `inlet_violation_steps ≤ inlet_tol_steps` (default tolerance 0 — zero violations).
- 3 `inlet_temp_max + inlet_forecast_margin ≤ inlet_cap` (default cap 26.0 °C).
- 4 If `rh_hard == True`: `rh_violation_steps ≤ rh_tol_steps` (off by default).

Key files.

Symbol	Location	Role
<code>BeamPlanner.plan</code>	<code>src/planner/beam_search.py:85</code>	Entry point; returns <code>PlanResult</code> (best setpoints, score, evals, feasible, history, finalists).

Symbol	Location	Role
<code>BeamConfig</code>	<code>src/planner/beam_search.py:33</code>	Hyperparameters: <code>grid=5</code> , <code>beam_width=5</code> , <code>levels=3</code> , <code>neighbors=8</code> , <code>max_evals=400</code> , <code>epsilon=1e-3</code> .
<code>_coarse_grid</code>	<code>src/planner/beam_search.py:60</code>	<code>np.linspace</code> Cartesian product → g^3 candidates.
<code>score / is_feasible</code>	<code>src/planner/objective.py:47 / :35</code>	Scalar objective + hard gate.
<code>ObjectiveWeights</code>	<code>src/planner/objective.py:12</code>	Soft λ weights + hard tolerances (<code>inlet_cap=26.0</code> , <code>inlet_forecast_margin</code> , <code>rh_hard</code>).
<code>DEFAULT_SEARCH_SPACE</code>	<code>src/planner/types.py:81</code>	SAT [20,26], flow [4.8,13.8], CHWST [13,19].
<code>run_weekly_plan</code>	<code>src/planner/pipeline.py:62</code>	Orchestration.

Configure / extend. Tune the search via `PlanRequest` (`grid`, `beam_width`, `levels` map directly to `BeamConfig`). Tune the objective via `ObjectiveWeights`. There is no callback slot for a custom objective today — to change the energy term, modify `score()` or have the oracle set a different `energy_term` (e.g. via tariff). See [Extending DCTwin](#).

Termination & edge cases. Hard cap at `max_evals`; soft early-stop when the best score improves by less than `epsilon · max(|prev|, 1)`. If the coarse grid shows < 0.1 °C inlet spread **and** < 1 % energy spread, `PlanResult.degenerate_no_signal=True` flags a control-invariant model (any "winner" is noise). If g^3 exceeds `max_evals`, the grid is **lexicographic-stride** subsampled (not random) — the first dimension stays fully sampled, the last is sparse.

5.2 EnergyPlus oracle & Docker

What it does. Scores each candidate with a real full-week EnergyPlus 9.5 simulation, fanned out across **processes** (the `dctwin` config is a process-global singleton, so threads would corrupt each other's case directories).

Flow per candidate (`evaluate_one`, `src/planner/oracle_worker.py:168`):

- 1 `dctwin.make_env(env_proto_config=week_config_path)` builds a gym `EPlusEnv` (`dctwin/gym_envs/eplusr_env.py:18`) wrapping `EplusDockerBackend`.
- 2 `_configure_backend` points the EnergyPlus container at the reachable BCVTB host and sets the socket timeout to the per-candidate budget.
- 3 `mapper_from_env` derives the `BroadcastPolicy`; `broadcaster.expand(Setpoints)` maps 3 globals → N normalized actions.
- 4 `discover_monitor` introspects `env.observations` to find power/thermal/HVAC sensors and scope them to the controlled hall.
- 5 `run_episode` loops `env.step` to the end of the week, collecting `StepSamples`; `aggregate_kpi` produces the `WeeklyKPI`.
- 6 A watchdog (`_run_with_timeout`) kills the container on hang to break a blocked `recv()`.

Co-simulation. EnergyPlus runs in `ghcr.io/cap-dcwiz/energyplus-9-5-0:latest`; the worker opens a listening socket, writes `socket.cfg/variables.cfg`, runs the container (`DockerBackend.run_container`), and exchanges actions/observations over BCVTB each step.

Key files.

Symbol	Location	Role
<code>ParallelEnvOracle.evaluate</code>	<code>src/planner/oracle.py:107</code>	Public API; builds an <code>EvalTask</code> per candidate, fans out across a <code>ProcessPoolExecutor</code> , collects with a stall guard.
<code>OracleConfig</code>	<code>src/planner/oracle.py:19</code>	<code>n_workers=8, timeout_s=300, timesteps_per_hour=4, bcvtb_host="172.17.0.1", monitored_hall="1f 2a".</code>
<code>evaluate_one</code>	<code>src/planner/oracle_worker.py:168</code>	Per-worker episode.
<code>aggregate_kpi</code>	<code>src/planner/kpi.py:50</code>	Aggregates post-warmup samples into <code>WeeklyKPI</code> .
<code>discover_monitor</code>	<code>src/planner/monitor.py:35</code>	Sensor discovery + hall scoping.
<code>make_env</code>	<code>dctwin/registration.py:9</code>	prototxt → gym env factory.
<code>EplusBackendMixin</code>	<code>dctwin/third_parties/eplus/core.py:23</code>	BCVTB protocol.
<code>MockEvaluator</code>	<code>src/planner/mock_evaluator.py:24</code>	Fast analytic stand-in for all unit tests (no Docker).

Configure / extend. `OracleConfig` controls parallelism and timeouts. **Gotchas to respect:**

Docker volume mount paths must be absolute; `bcvtb_host` must be the Docker0 gateway `172.17.0.1` on Linux (not `host.docker.internal`) and set **before** `env.reset()`; the default socket timeout is 1 hour and must be overridden per candidate; `aggregate_kpi` skips the first 6 warmup steps; a week that crosses a calendar-year boundary raises `ValueError` (EnergyPlus hardcodes year 2013).

5.3 Candidate generation

What it does. Enumerates the candidates the oracle scores, and expands the chosen 3-tuple to the env's 45 actuators.

- **Coarse grid:** `_coarse_grid` (`beam_search.py:60`) produces `grid3` candidates (e.g. $5^3 = 125$) via `np.linspace` per dimension.
- **Neighbors:** `_neighborhood` (`beam_search.py:174`) emits up to 8 local offsets ($\pm$ step per dim plus 2D diagonals), clipped to bounds by `SearchSpace.clip` (`types.py:40`). The step halves each refinement level.
- **Broadcast:** `BroadcastPolicy.expand` (`broadcast.py:46`) maps `Setpoints(sat, flow, chwst)` to an N-dim `[-1,1]` vector via `normalize(x, lb, ub) = 2 * (x-lb) / (ub-lb) - 1`. The GDS hall is **45 AGENT_CONTROLLED actuators in fixed order [22 SAT, 22 FLOW, 1 CHWST]** (`gds_action_spec, broadcast.py:59`). A single SAT value replicates to all 22 SAT actuators; same for FLOW; CHWST is global. **This order is load-bearing** — a prototxt reordering silently breaks the mapping.

Time-block (day/night) scheduling (opt-in via `PlanRequest.time_block`):

- `TimeBlock/WeeklySchedule` (`src/planner/schedule.py`) model per-block constant setpoints with midnight wraparound (`DEFAULT_BLOCKS`: day 6–18, night 18–6).
- `refine_schedule` (`src/planner/schedule_search.py:35`) is a 2-level warm-start coordinate descent **seeded at the constant optimum** — guaranteeing the result is never worse than the constant.

- `evaluate_one_schedule` (`oracle_worker.py:245`) switches the action per simulation hour via `schedule.block_for_hour`.
- The top-level `recommendation.setpoints` mirror the day block for backward compatibility; the full schedule lives in `recommendation["schedule"]` only when `time_block=True`.

5.4 Robust "three-net" safety gate

The hard inlet ≤ 26 °C cap binds across **three independent safety layers**, each hedging a distinct uncertainty, with single-counted allocation (no double-hedging).

Net	Hedges	Mechanism	Location
1 — $k \cdot \sigma$ pre-tightening	twin-vs-plant model error	Search treats the cap as $26 - k \cdot \sigma$. <code>apply_forecast_margin</code> sets <code>inlet_forecast_margin = K_SIGMA · σ_{inlet} ($K_SIGMA=1.0$);</code> candidates breaching the tightened cap score <code>+inf</code> . Cold-start $\sigma =$ <code>SIGMA_PRIOR=1.0 °C</code> → search cap 25.0 °C.	<code>src/planner/pipeline.py:19</code> , <code>objective.py:40</code>
2 — robust scenario ensemble	plant degradation (fan/coil fouling, wear)	Each finalist is re-simulated across N perturbed-plant scenarios (default 4) whose factors scale by <code>[1-spread, 1+spread]</code> . Each scenario KPI is bias-corrected then tested against the <i>hard</i> cap with <code>inlet_forecast_margin=0</code> (no stacking on net 1). A finalist is robust-feasible iff $\geq \lceil N/2 \rceil$ scenarios succeed and all successful ones pass.	<code>src/planner/robust.py:173</code> , <code>robust_select:132</code>
3 — safety-ladder backstop	energy-optimality traps & discrete failures	If all finalists are fragile, enumerate the energy↔robustness frontier (<code>safety_ladder</code> : cheap axes first — CHWST↓, then SAT↓, then full diagonal to the max-cooling corner) and recommend the cheapest provably-robust variant. Only if the max-cooling corner itself fails does status become <code>blocked_unsafe</code> .	<code>src/planner/robust.py:64</code> , <code>pipeline.py:129</code>

Ensemble sizing. `scenario_spread` (`robust.py:44`) scales `base_spread=0.1` by the empirical-Bayes posterior $\sigma_{post} / \sigma_{ref}$, floored at `MIN_SPREAD=0.02` (the ensemble never collapses below ± 2 % drift). `$\sigma_{post} = \sqrt{(n \cdot s^2 + \sigma_{prior}^2) / (n+1)}$` — the prior counts as one pseudo-week, so even one accurate week buys only a $\sqrt{2}$ tightening, never a collapse.

Configure / extend. `K_SIGMA`, `SIGMA_PRIOR`, `MIN_SPREAD`, `n_scenarios`, and `inlet_cap=26.0` are the levers. The hard cap is a non-negotiable invariant — **do not weaken these gates to make a**

search succeed. Status codes: `pending_approval` (feasible), `blocked_unsafe` (robust-infeasible even after the ladder), `infeasible_fallback` (no feasible candidate under nominal).

The outer deployment loop

6.1 Pre-validation

What it does. A post-planning, pre-deployment independent replay that re-evaluates the recommended setpoints with a fresh oracle (not the stored `predicted_kpis`) to catch model drift. It runs immediately after planning **inside** `run_plan_job`, but is **advisory — it never fails the plan** (any exception is logged; the plan still reaches `pending_approval`).

`run_prevalidation_with_oracle` (`src/prevalidation.py:68`) runs two scenarios in parallel: a **nominal** replay and a **worst-case** (most-degraded plant from the calibration spread). It compares against a conservative baseline (coolest SAT, max flow, coolest CHW). Verdict: `passes = (inlet_violations == 0) AND (energy_reduction > 0 %)` (`validation_metrics`, `src/planner/validation.py:7`).

Artifacts (`runs/<id>/prevalidation/`): `report.md` (PASS/FAIL + KPI table), `trajectory_ai.csv` and `trajectory_worst.csv` (15-min inlet/power/PUE series), plus `oracle/` and `worst/` simulation logs.

Gotchas. Setpoint edits set `needs_revalidation=true` and clear `predicted_kpis` but do **not** auto-rerun prevalidation — the expert must run a fresh plan for updated assurance. Approval is blocked while `needs_revalidation=true`. Trajectory CSVs are always 15-min granularity regardless of the planning resolution.

6.2 Shadow-mode deployment

What it does. `deploy()` (`src/deploy.py:28`) is the approval-guarded backstop. It raises `PermissionError` unless `rec["status"] == "approved"`, then (when a BMS adapter is passed) calls `bms.apply()` and runs the perturbed-plant oracle to record realized KPIs.

The **BMS seam** (`src/planner/bms.py`) has two interfaces:

- `ShadowBmsAdapter.apply` (`bms.py:52`): expands the 3 setpoints into **45 denormalized commands** in GDS order, writes `bms_commands.json` (`mode:"shadow", actuated:false`), and **never actuates** — it only audits.
- `BacnetBmsAdapter` (`bms.py:73`): the field seam, raises `NotImplementedError` documenting the required site config (BACnet host, 45-point device map, write priority).

`bms_adapter_for_mode` (`jobs.py:331`) reads `DTWIN_DEPLOY_MODE` (webapp default `"shadow"`); `"sim"` returns `None` (pre-1.8 pure-sim path). The realized week always comes from the oracle (`realized_source:"sim"`) so calibration keeps learning. Schema **1.8** adds `deploy_mode`, `bms`, `realized_source` additively. The hard 0-tolerance cap applies one final time: any realized inlet violation flips status to `deploy_blocked`.

Live telemetry & recirculation ship alongside deployment: `TelemetryStore` (SQLite, `telemetry.py:48`), `SimTelemetryFeed` (daemon, 22 rack inlets + power/PUE/held setpoints every 5 s, always labelled `simulated=1.0`), the `RecircAwareEvaluator` wrapper (`recirc.py:81`, opt-in when `data/recirc.json` calibrates `demand_kg_s > flow.lb`), and `fit_recirc.py` to fit the recirculation fraction `r0` from telemetry.

6.3 Physics recalibration

What it does. Closes the fidelity loop after each deploy. Two complementary mechanisms:

- **Output-residual calibration** (`calibrator.py`): fits per-KPI additive `bias` (mean of winsorized residuals) and uncertainty `sigma` (fading-floor `max(sample, prior/n)`) + `sigma_post` (empirical-Bayes) from the paired deploy history. `Calibration.apply` (`calibrator.py:43`) corrects each candidate's KPI before scoring, and flags an inlet violation if a corrected inlet exceeds 26 °C. `sigma_for` backs the net-1 margin; `sigma_post_for` sizes the net-2 ensemble.
- **Physics recalibration** (`recalibrator.py`): maps a persistent energy bias to a fan-efficiency factor. `fit_plant_factors` (`recalibrator.py:23`) computes `b = mean(realized/predicted ratios) - 1`, requires ≥ 4 weeks and $|b| \geq 0.01$, and proposes `factor = clip(1/(1+b), 0.85, 1.15)`. `recalibrate` (`recalibrator.py:48`) writes a perturbation proposal merged over `DEFAULT_PLANT` (fan 0.93×, coil 0.85×) by `load_plant_config` to recenter the next plan's robust ensemble.

Wiring (`run_deploy_job`, `jobs.py:344`): after the plant week → `advance_calibration` (append paired predicted/realized) → `recompute_calibration` (refit + persist `calibration.json`) → `write_plant_calibration` (guarded; writes `plant_calibration.json` if drift is actionable). The next plan loads these so its search is bias-corrected and its ensemble centers tighter.

Invariants. Residuals are fit against **raw** predictions (`residual_predicted_for`, `jobs.py:296`) to avoid double-correction; recalibration is fully `try/except`-wrapped so a hiccup never fails the deploy; the fading floor never collapses to $\sigma=0$.

6.4 As-operated baseline

What it does. Derives the plant's current control state from telemetry so savings are measured against reality, not a synthetic point. `as_operated_setpoints` (`src/planner/baseline.py:41`) regex-matches multiple CRAH/chiller columns, takes the pooled nanmedian per axis, converts median fan-speed to flow (`flow = (median_fan / fan_speed_max) · design_flow_kg_s_per_acu`), and clips to the search space (silently falling back to mid-range when a signal is absent).

`run_weekly_plan` evaluates the baseline **exactly once** per planning run (`pipeline.py:110`) on the same forecast and calibration as the AI plan, so the energy-reduction comparison is honest. Schema **1.7** records the `baseline` block (`source:"as_operated"`, `energy_kwh`, `setpoints`, `kpis`) and the `energy_scope` label. Loop closure writes two **separate, incompatible** files: `realized_history.csv` (weekly KPI summary) and `calibration_history.json` (paired predicted/realized for residual fitting) — never mix them with the forecaster's per-step IT-load CSV.

Forecasters

The forecasters supply the workload + weather the oracle simulates against. Both are honest naive models (no external provider), reflecting genuine short-horizon uncertainty.

IT-load forecaster (`src/planner/forecaster.py`):

- `loading_from_it_loads` normalizes per-hall kW to a 0–1 CPU-loading fraction.
- **Persistence** (`StatisticalForecaster`, :134): last-`n_steps` of the loading series, no calendar awareness.
- **Seasonal** (`SeasonalForecaster`, :167): a (weekday × time-of-day) climatology with p10/p50/p90 bands, re-leveled to the calendar period via `calendar_level_scale` (mean in `week_start ± 10 days` / mean of all, clipped [0.5, 1.5]). Falls back to hourly then global percentiles when a bucket has < 4 samples.
- The 1F-2A load is empirically **flat** (mean ~971 kW, span 870.8–1057.9 over ~116 days), so the forecast appears nearly constant — that is **correct**, not a bug. Week-to-week variation is weather-driven. Only the seasonal method populates `forecast.bands`.

Weather forecaster (`src/planner/weather_forecast.py`, `epw.py`):

- `weather_stats` (:33) extracts the **historical-analog spread** — dry-bulb mean/ σ for the target week's month-days (year-agnostic, ± 7 days) from the EPW. This is the swap-in seam for a real NWP provider: replace this one function's `{mean_c, sigma_c, n}` return and everything downstream is unchanged.
- `weather_scenarios` (:95) generates nominal/hot/cool EPW variants by shifting dry-bulb $\pm k \cdot \sigma$ ($k=1.0$), clamping dew-point $\leq$ shifted dry-bulb. The robust gate's hot-weather scenario uses the hot variant.
- `week_within_epw` validates the week fits the EPW coverage; a year-boundary week raises `ValueError` at `write_week_config`.

Fit & backtest. `fit_forecaster.py:main` pickles `models/forecaster.pkl` (`{method, his_csv, room2ite_path, his_col_for_room, weather_file}`). `backtest_forecaster.py` reports RMSE/MAPE/PICP on a held-out window (calibrated bands give $\text{PICP} \approx 0.8$).

Oracle integration. The oracle calls `forecast.materialize()` to write per-ITE workload schedules, then `write_week_config(weather_file=forecast.weather_file)` to set the EnergyPlus RunPeriod and weather, optionally lifting ACU masking so the planner can freely optimize.

Web application & operator workflow

A single-origin **FastAPI** backend (port 8000) serves a **React 19 + Vite + TypeScript** frontend built into `dist/` and mounted at `/`. Auth is Bearer-token with two roles: **operator** (creates plans, monitors) and **expert** (approves, edits, deploys). `TokenAuth` (`auth.py:11`) is fail-closed (denies if no tokens configured) unless `DTWIN_INSECURE=1`.

Backend routes (selected):

Method & path	Role	Handler	Purpose
POST <code>/api/plans</code>	operator	<code>create_plan</code> (<code>main.py:146</code>)	Create plan, enqueue job.
GET <code>/api/plans/{id}</code>	operator	—	List / detail.
GET <code>/api/plans/{id}/stream</code>	operator (token in query)	<code>plan_sse_stream</code> (<code>main.py:42</code>)	SSE progress every 0.5 s, keepalive every 15 s.
POST <code>/api/plans/{id}/approve</code>	expert	<code>approve</code> (<code>main.py:222</code>)	Guarded by state machine + <code>needs_revalidation</code> .
PATCH <code>/api/plans/{id}/setpoints</code>	expert	<code>edit_setpoints</code> (<code>main.py:287</code>)	Set <code>needs_revalidation</code> , clear predicted KPIs.
POST <code>/api/plans/{id}/deploy</code>	expert	<code>deploy_plan</code> (<code>main.py:268</code>)	Set <code>DEPLOYING</code> before enqueue (idempotent).
GET <code>/api/live/{stream, /series}</code>	operator	telemetry	Live frame, SSE, time-series.
POST <code>/api/telemetry</code>	operator	<code>post_telemetry</code> (<code>main.py:414</code>)	Real-historian push seam.
GET <code>/api/topology</code>	operator	<code>build_hall_topology</code> (<code>topology.py:224</code>)	3D scene layout.

JobRunner & SSE. `JobRunner` (`jobs.py:25`) is a single-worker thread queue. `run_plan_job` (`jobs.py:142`) builds the oracle/forecaster/calibration, runs `run_weekly_plan`, writes `recommendation.json` + `progress.json`, then runs prevalidation. `run_deploy_job` (`jobs.py:344`) runs the perturbed plant, persists realized KPIs, advances history/calibration. Cancellation is

cooperative (`progress_cb` raises `PlanCancelled`); orphans are reconciled to terminal on startup.

Status state machine (`status.py`): `queued` → `running` → `{pending_approval | infeasible_fallback | blocked_unsafe | failed | cancelled}`; `pending_approval` → `{approved | rejected}`; `blocked_unsafe/infeasible_fallback` → `rejected` only; `approved` → `{deploying | rejected}`; `deploying` → `{deployed | deploy_blocked | deploy_failed}`. `can_transition` (`status.py:35`) gates every change.

Frontend pages (`src/frontend/src/pages/`): **Login**, **Dashboard** (latest plan + KPIs), **NewPlan** (form + live SSE progress, 5× reconnect), **Review** (setpoints, predicted KPIs, baseline, robust bands, trajectory, expert actions), **Live** (22-rack heat-map, alerts at $\geq 25/\geq 26$ °C, compliance ± 0.5 /axis, 30-min chart, SSE + poll fallback), **History** (sortable/filterable table), **DigitalTwin3D** (three.js hall + animated airflow, live rack-row coloring). The API client is `src/api.ts`; EventSource URLs carry the token in the query (`planStreamUrl`, `api.ts:112`) since EventSource cannot set headers.

Run / build / test commands.

```
# whole app, single origin
scripts/clear-and-run.sh

# frontend dev (hot reload, proxies /api → :8000)
env -C /mnt/lv/home/hoanghuy/newcode/dctwin/src/frontend npm run dev
env -C /mnt/lv/home/hoanghuy/newcode/dctwin/src/frontend npm run build # tsc -b && vite build
env -C /mnt/lv/home/hoanghuy/newcode/dctwin/src/frontend npm test # vitest
```

Notable gotchas. Progress writes are atomic (`os.replace`); non-finite floats are replaced with `null` before JSON serialization; deploy is idempotent (status set to `DEPLOYING` before enqueue, `run_deploy_sync` skips already-deployed); the inlet 26 °C cap is a 0-tolerance deploy gate.

Experimental results

The repo captures **6 consecutive weekly cycles** (2024-11-08 → 2024-12-13) under `src/runs/gds-2024-MM-DD-HASH/`.

Run-directory layout.

File / folder	Contents
<code>recommendation.json</code> (schema 1.7–1.8)	Setpoints, <code>predicted_kpis</code> , <code>baseline</code> , <code>robust</code> (confidence bands p50/p90/max, scenario diagnostics), <code>status</code> , <code>k_sigma</code> , <code>inlet_forecast_margin</code> , <code>energy_scope</code> .
<code>realized.json</code> (schema 1.8)	Measured <code>total_hvac_energy_kwh</code> , <code>pue_mean</code> , <code>inlet_temp_max_c</code> , <code>inlet_violation_steps</code> .
<code>progress.json</code>	Search checkpoint (level, evals, best score).
<code>oracle/cand-XXXX/.../eplusout.csv</code>	Per-candidate full-week EnergyPlus output (6 MB+, 15-min timeseries).
<code>robust/scenario-00...03/</code>	Robust re-sims (4 perturbed plants + hot-weather), each with <code>plant.idf</code> + <code>oracle/</code> .
<code>prevalidation/</code>	<code>report.md</code> , <code>trajectory_ai.csv</code> , <code>trajectory_worst.csv</code> .
<code>deploy/oracle/</code>	Deployed-week run (ground truth for calibration); <code>bms_commands.json</code> records the 45 shadow commands.
<code>plant/</code>	Physics model snapshot (<code>plant.idf</code> , <code>plant.prototxt</code>) frozen at deploy time.

Where the numbers live. Predicted energy: `recommendation.json` → `predicted_kpis`. Realized energy/inlet: `realized.json`. Savings: `energy_reduction_vs_baseline_pct`. Safety: `realized.json` → `inlet_temp_max_c` / `inlet_violation_steps`. Calibration σ : `recommendation.json` calibration fields.

Figures (`dctwin/BHUMANS/deck_assets/`, generated by `make_deck_full.py`):

Figure	Shows
<code>figA_accuracy.png</code>	Predicted vs realized energy across 6 weeks; prediction error shrinking.
<code>figB_savings.png</code>	Energy savings ramp (0 % → ~3.6 %) correlated to uncertainty σ dropping (1.0 °C → ~0.43 °C).
<code>figC_safety.png</code>	Realized peak inlet (23.37 → 25.60 °C) vs the 26 °C hard cap; 0 violations.
<code>figD_safety_stack.png</code>	The three-net safety mechanism + safety-ladder substitution.
<code>figE_dataflow.png</code>	Closing-the-loop dataflow (shadow BMS, telemetry, live dashboard, field seams).
<code>figF_procedure.png</code>	The 6-step operator weekly procedure.

The headline measured results and forensic findings are summarized in `dctwin/BHUMANS/review_after_4th_draft.md` (\$2.4) and the dated findings under `docs/superpowers/specs/`.

Caveat (important for citing results). Realized KPIs come from the **perturbed-plant EnergyPlus simulation**, not real field telemetry — the loop is *structurally* closed, but "reality" is still a physics simulation. Recirculation is currently **inert** (the `AdjustedSupply` blend sees zone air $\approx$ supply temp), so inlet is driven by SAT + flow-dependent fan heat; this makes the safety margin optimistic versus a real hall with imperfect containment. Always check `recommendation.json` schema is ≥ 1.7 before comparing baseline/savings fields.

Extending DCTwin (developers)

DCTwin follows a strict workflow: **brainstorm** → **spec** (`docs/superpowers/specs/`) → **plan** (`docs/superpowers/plans/`) → **adversarial verification** → **subagent build** → **merge** (`--no-ff`) → **graphify update**. Plans are written **test-first**; never weaken an assertion to make code pass.

Recipe: add a new objective

The objective is hard-coded in `score()` (`objective.py:47`), not a callback. Either (a) modify `score()` to add/replace a term, or (b) have the oracle set a different `energy_term` on the `WeeklyKPI` (the tariff path is the canonical example — `weighted_energy_cost` replaces raw energy when `data/tariff.json` exists). Bump the schema version if the recommendation shape changes.

Recipe: change the search space

`DEFAULT_SEARCH_SPACE` (`types.py:81`) is the single source of truth. There is no JSON config for bounds — construct a custom `SearchSpace` and pass it to `BeamPlanner.__init__`. Per-actuator bounds can come from the live env via `mapper_from_env`. If you change the 45-actuator order, update `gds_action_spec` (`broadcast.py:59`) **and** the prototxt declaration **and** `expand_commands` in `bms.py` — they must agree.

Recipe: add a constraint

Add a hard check to `is_feasible()` (`objective.py:35`) and/or a soft penalty term to `score()`. Soft-penalty accumulators must be **computed by the oracle** (`aggregate_kpi`, `kpi.py:50`) and attached to `WeeklyKPI` (`types.py:49`) — the objective only sums them. Respect the conjunctive hard gate and the 26 °C invariant.

Recipe: plug a real BMS

Implement `BacnetBmsAdapter.apply` (`bms.py:73`) with the same contract as `ShadowBmsAdapter` but `actuated:true`. Wire it through `bms_adapter_for_mode` (`jobs.py:331`) behind a new `DTWIN_DEPLOY_MODE` value. Verify the 45-command order matches `gds_action_spec` and the prototxt.

Recipe: add an API route / frontend page

Backend: add the route in `webapp/main.py`, guard it with the appropriate role via `TokenAuth`, and respect the status state machine (`can_transition`). Frontend: add a page under `src/frontend/src/pages/`, a client call in `src/api.ts`, and a nav entry in `App.tsx`. The build (`tsc -b`) type-checks test files with `noUnusedLocals` ON — drop unused imports or the build fails (TS6133).

Run the tests

```
# fast unit tests (EnergyPlus mocked via MockEvaluator)
env -C /mnt/lv/home/hoanghuy/newcode/dctwin/src \
  /mnt/lv/home/hoanghuy/newcode/dctwin/.venv-dtwin/bin/python -m pytest -q

# Docker-gated integration tests (real EnergyPlus; slow, flaky on BCVTB – wrap in a hard timeout)
sg docker -c "env -C /mnt/lv/home/hoanghuy/newcode/dctwin/src PYTHONPATH=\$PWD \
  /mnt/lv/home/hoanghuy/newcode/dctwin/.venv-dtwin/bin/python -m pytest -m integration -q"

# frontend
env -C /mnt/lv/home/hoanghuy/newcode/dctwin/src/frontend npm test
```

`pyproject.toml` sets `addopts = "-q -m 'not integration'"`, so Docker tests are deselected by default. Rely on the exit code + `N passed` count (the summary is often buried under warnings).

Key invariants you must not break

- **Inlet ≤ 26 °C at every step** — enforced in search (net 1), scenarios (net 2), and deploy backstop. Never relax it to make a search succeed.
- **Process-based parallelism** — the `dctwin` config is a process-global singleton; never assume thread-safety for config-dependent state.
- **No double-hedging** — scenarios clear `inlet_forecast_margin` (`robust.py:201`); don't stack net-1's $k\cdot\sigma$ inside net-2.
- **Schema is versioned and additive** — bump on shape changes (`1.0 → 1.7 → 1.8`), keep older readers working.
- **Residuals fit against raw predictions** — `residual_predicted_for` prevents double-correction.
- **Don't weaken tests** — implementers make code pass; never relax an assertion.
- After merging code, **regenerate** `graphify-out/`. Do **not** push to `origin` without explicit go-ahead.

Glossary & key conventions

Term	Meaning
SAT	CRAH supply-air temperature setpoint (°C), bounds [20, 26].
Flow / airflow	CRAH supply-air mass flow per ACU (kg/s), bounds [4.8, 13.8].
CHWST	Chilled-water supply temperature (°C), bounds [13, 19].
Oracle	The full-week EnergyPlus-in-Docker evaluator (ParallelEnvOracle) that scores each candidate.
Broadcast	Expansion of the 3 global setpoints to the 45-actuator [22 SAT, 22 FLOW, 1 CHWST] normalized action vector.
energy_scope	Label (" hall_controllable_v1 ") for the HVAC energy metric: the 1F-2A ACU fans + shared chiller/CHW plant — not facility total-IT (which is the legacy fallback only when component powers aren't discovered).
WeeklyKPI	The aggregated 1-week outcome (energy, PUE, inlet max, violations, soft penalties, optional weighted cost).
Three nets	The layered safety gate: $k\cdot\sigma$ pre-tightening, robust ensemble, safety-ladder deploy backstop.
Shadow mode	Deployment that writes 45 BMS commands to disk but never actuates; realized KPIs still come from the oracle.
Schema versioning	recommendation.json versions: 1.0 → 1.5 (adds time-block schedule) → 1.7 (adds as-operated baseline + energy_scope) → 1.8 (adds deploy_mode/bms/realized_source).
mock_evaluator	The fast analytic stand-in for EnergyPlus used by all unit tests — never invoke real EnergyPlus from a unit test.
Degenerate / no-signal	Coarse grid with < 0.1 °C inlet spread and < 1 % energy spread → control-invariant model, flagged for review.

Reference map

Concern	Primary file(s):line
Beam search algorithm	src/planner/beam_search.py:85 (plan), :33 (BeamConfig), :60 (_coarse_grid), :174 (_neighborhood)
Objective & feasibility	src/planner/objective.py:47 (score), :35 (is_feasible), :12 (ObjectiveWeights)
Setpoint / search-space types	src/planner/types.py:8 (Setpoints), :35 (SearchSpace), :81 (DEFAULT_SEARCH_SPACE), :49 (WeeklyKPI)
Pipeline orchestration	src/planner/pipeline.py:62 (run_weekly_plan), :19 (apply_forecast_margin)
EnergyPlus oracle	src/planner/oracle.py:107 (evaluate), :19 (OracleConfig); src/planner/oracle_worker.py:168 (evaluate_one)
KPI aggregation / monitor	src/planner/kpi.py:50 (aggregate_kpi); src/planner/monitor.py:35 (discover_monitor)

Concern	Primary file(s):line
EnergyPlus / BCVTB / Docker	dctwin/registration.py:9 (make_env); dctwin/third_parties/eplus/core.py:23; dctwin/third_parties/docker_backend.py:14
Mock evaluator (tests)	src/planner/mock_evaluator.py:24
Broadcast (3 → 45)	src/planner/broadcast.py:46 (expand), :59 (gds_action_spec)
Time-block scheduling	src/planner/schedule.py:32; src/planner/schedule_search.py:35 (refine_schedule)
Robust "three-net" gate	src/planner/robust.py:173 (make_oracle_robust_rerank), :132 (robust_select), :64 (safety_ladder), :44 (scenario_spread)
Plant perturbation	src/planner/plant.py:21 (PlantConfig), :28 (DEFAULT_PLANT), :58 (apply_perturbation)
Pre-validation	src/prevalidation.py:68 (run_prevalidation_with_oracle); src/planner/validation.py:7 (validation_metrics)
Deploy + BMS seam	src/deploy.py:28 (deploy); src/planner/bms.py:52 (ShadowBmsAdapter.apply), :73 (BacnetBmsAdapter)
Calibration	src/planner/calibrator.py:81 (fit_calibration), :43 (apply), :122 (recompute_calibration)
Physics recalibration	src/planner/recalibrator.py:23 (fit_plant_factors), :48 (recalibrate)
As-operated baseline	src/planner/baseline.py:41 (as_operated_setpoints); src/planner/recommendation.py:11 (energy_reduction_pct)
Loop closure / history	src/planner/history.py:16 (advance_history), :46 (advance_calibration)
IT-load forecaster	src/planner/forecaster.py:134 (StatisticalForecaster), :167 (SeasonalForecaster), :208 (build_forecaster)
Weather forecaster	src/planner/weather_forecast.py:33 (weather_stats), :95 (weather_scenarios); src/planner/epw.py:50 (week_within_epw)
Forecaster fit / backtest	src/fit_forecaster.py:42 (main); src/planner/backtest_forecaster.py:31
Week config (prototxt)	src/planner/week_config.py:32 (write_week_config)
Live telemetry / recirc	src/webapp/telemetry.py:48 (TelemetryStore), :126 (SimTelemetryFeed); src/planner/recirc.py:81 (RecircAwareEvaluator); src/fit_recirc.py:75
Web backend	src/webapp/main.py:102 (create_app), :42 (plan_sse_stream); src/webapp/jobs.py:25 (JobRunner), :142 (run_plan_job), :344 (run_deploy_job)
Auth / status / store	src/webapp/auth.py:11 (TokenAuth); src/webapp/status.py:35 (can_transition); src/webapp/store.py:31 (PlanStore)

Concern	Primary file(s):line
Frontend shell / client	src/frontend/src/App.tsx:23; src/frontend/src/api.ts
Frontend pages	src/frontend/src/pages/{Login,Dashboard,NewPlan,Review,Live,History,DigitalTwin3D}.tsx
3D topology / scene	src/webapp/topology.py:224 (build_hall_topology); src/frontend/src/three/HallScene.tsx:61
Result artifacts	src/runs/gds-2024-MM-DD-HASH/{recommendation,realized,progress}.json, oracle/, robust/, prevalidation/, deploy/, plant/
Report figures	dctwin/BHUMANS/deck_assets/fig{A,B,C,D,E,F}.png; dctwin/BHUMANS/make_deck_full.py